Laboratory 1: Short project

Docker, Compose and Swarm — high availability edition

Marc De Rialp David Franco Omar Antonio Cornejo David Madrid
Victor Yakovlev

Cloud Computing and Distributed Systems
MSc in Machine Learning and Cybersecurity for Internet Connected Systems, UPC

October 2026

Abstract

We containerised the digital infrastructure of a company with Docker: a shared stack (web front, certificates, shared disk), a reviews application with its database, cache, mail relay and backups, a monitoring platform, and the Moodle learning management system. The same project runs in two environments: a local one on a single machine (`compose.yml`) and a production one on a Docker Swarm of seven nodes (`production.yml`), in which every user-facing service is replicated, the database and the cache are clustered and fail over by themselves, and the backups are encrypted and kept on a node of their own. The images are published in Docker Hub, so deploying only needs to pull and start them. The report describes the design, the deployment, the management of data and backups, the monitoring, the test cases and their results.

Contents

1 Introduction **3**

2 Architecture **3**
2.1 Nodes . . . 3
2.2 Services . . . 3

3 Images and design decisions **4**

4 Scalability and high availability **5**
4.1 Replicas . . . 5
4.2 Failures . . . 6
4.3 Updates . . . 6

5 Deployment **6**

6 Access, users and passwords **7**
6.1 Web addresses . . . 7
6.2 Users and passwords . . . 7

7 Data management and backups **7**

8 Observability **8**

9 Security **9**

10 Test cases **9**
10.1 Local environment . . . 9
10.2 Production environment . . . 10

11 Results **11**
11.1 Local environment . . . 11
11.2 Production environment (`production.yml`) . . . 11

12 Limitations **11**

13 Conclusions **12**

1 Introduction

We containerised the infrastructure outlined in the laboratory assignment: the shared services (*basic*), a company application (*app*), an observability platform (*stats*), and Moodle (*atenea*). For the core application, we developed a reviews service in Flask: users can post comments with a 1-to-5 rating, view score distributions and recent reviews, trigger email notifications through an internal SMTP relay upon each submission, and ingest external ratings via an automated background feeder.

The implementation is designed for production-grade resilience: every user-facing component maintains at least two replicas distributed across distinct nodes, the application database and caching layers are clustered with automatic failover, backups are encrypted and stored on a dedicated node, and all services export health and telemetry metrics. Deployment is fully automated, requiring only a single `docker compose up` for local execution or `docker stack deploy` for production Swarm orchestration, using pre-built images published to Docker Hub.

The repository is organised into `compose.yml` (local), `production.yml` (Swarm), service directories containing their respective `Dockerfile` definitions and runtime assets, helper automation in `scripts/` (node labelling, image publishing, Swarm simulation, and disaster recovery), and this report. The local environment operates without TLS and uses named volumes, yet preserves the exact clustering topology present in production to enable verification on a single development machine.

2 Architecture

2.1 Nodes

The swarm comprises seven nodes; service placement is controlled via the Docker Swarm node label `role` (along with `slot` on the application nodes to enforce separation between primaries and replicas).

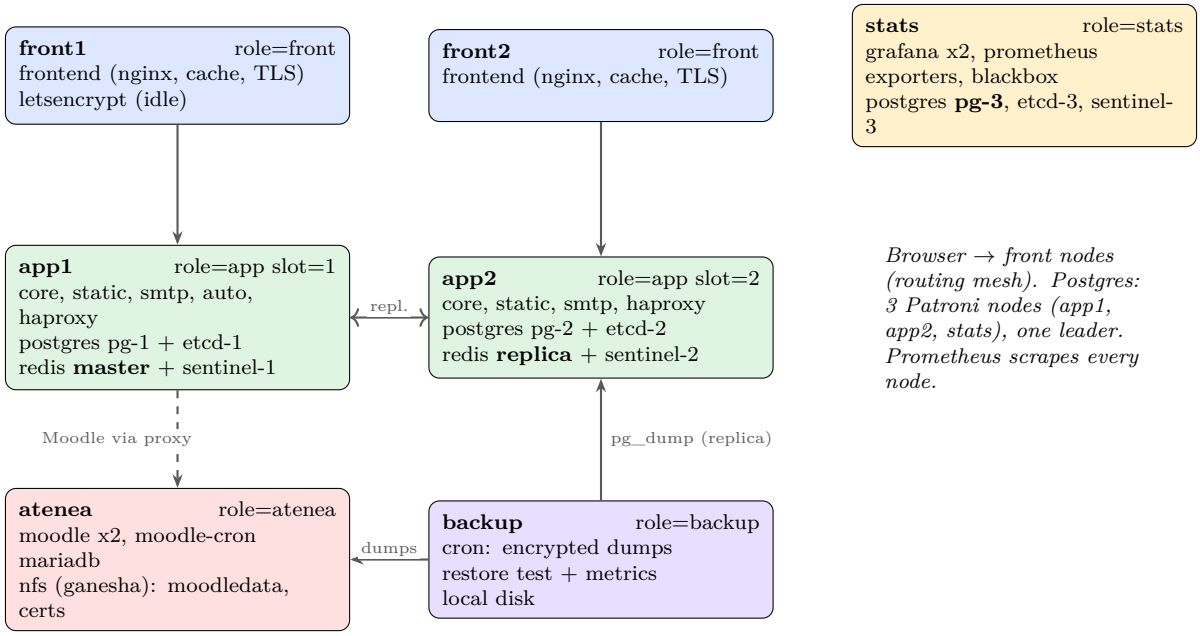


Figure 1: Nodes and services (simplified). The routing mesh accepts the traffic on both front nodes; `backup` pulls from a replica, from MariaDB and from the NFS volume.

2.2 Services

Table 1 maps the components from the assignment specification to our implemented services, the nodes where they run, and their replica counts. Each stack operates within an encrypted overlay

network, and each service only joins the networks it strictly requires: for instance, the databases do not participate in the `proxy` network, ensuring they remain isolated and inaccessible from the frontend.

Stack	Service	Nodes	Repl.	Description
basic	`frontend`	front	2	nginx: only public service (80/443), cache, TLS, proxy by subdomain
	`letsencrypt`	front	1	certbot; idle with a local domain (self-signed certificate)
	`nfs`	atenea	1	NFSv4 server (NFS-Ganesha) inside the stack, port 2049 in host mode
app	`core`	app	2	Flask + gunicorn, reviews
	`static`	app	2	Apache httpd, `/static/`
	`pg-1..3`	app1, app2, stats	3	PostgreSQL 16 + Patroni (leader + 2 replicas)
	`etcd-1..3`	app1, app2, stats	3	consensus store of Patroni
	`db`	app	2	HAProxy: `db:5432` leader, `db:5433` replicas
	`redis-1`, `redis-2`	app1, app2	1+1	cache: master and replica
	`sentinel-1..3`	app1, app2, stats	3	failover of the cache (quorum 2)
	`smtp`	app	2	Postfix relaying the mails (to Mailpit by default)
	`mailpit`	app	1	mail catcher with a web page (`mail.localhost:8080`)
	`auto`	app	1	feeder of external ratings
	`backup`	backup	1	encrypted dumps + restore test
atenea	`moodle`	atenea	2	Moodle 4.5.14 LTS
	`moodle-cron`	atenea	1	same image, `cron.php` every minute
	`moodle-db`	atenea	1	MariaDB 11.4
stats	`grafana`	stats	2	dashboards as code + Infinity plugin (the "?")
	`prometheus`	stats, backup	2	metrics and alert rules, one TSDB per replica
	exporters	stats / global	—	postgres, redis, mysqld, nginx, blackbox, node, cAdvisor

Table 1: Services of the infrastructure.

Public entrypoints in production are exposed via `app.<domain>`, `lms.<domain>`, and `grafana.<domain>`; with the default `DOMAIN=localhost`, they resolve to `app.localhost`, `lms.localhost`, and `grafana.localhost` on port 443. Locally, the same names are served on port 8080 over HTTP (modern browsers resolve `*.localhost` to 127.0.0.1 automatically).

3 Images and design decisions

Frontend and caching. We use `nginx:alpine` as the single public entrypoint for all services. Nginx terminates TLS, routes subdomains to their internal services, and caches dynamic pages for 10 seconds and static files for one hour. When a user submits a review, a temporary cookie bypasses the cache so their new comment appears immediately. The page footer displays which frontend and application nodes handled the request, demonstrating load distribution across replicas.

TLS and certificates. Production exposes HTTPS on port 443 using a self-signed certificate generated at container startup, allowing Nginx to boot without external dependencies. The `letsencrypt` service is fully defined in the stack with automated renewal logic, but remains idle when running with local domains (`localhost`) where public certificate authorities cannot validate ownership.

Core application. The reviews application is built with Python 3.12 and runs as a non-root user with two replicas. To maintain high performance, page data is cached in Redis with version-based invalidation upon new submissions. The application degrades gracefully: if Redis is unreachable, requests fall back directly to the database; if the database is temporarily recovering, the service returns a clean HTTP 503 instead of crashing. Database operations are decoupled via HAProxy: writes target the leader (port 5432) while reads are distributed across replicas (port 5433).

PostgreSQL cluster (Patroni). Database high availability is managed by Patroni across three PostgreSQL 16 nodes (`pg-1`, `pg-2`, `pg-3`) distributed over `app1`, `app2`, and `stats`. A three-node etcd cluster provides distributed consensus for leader election and health tracking. If the active leader fails, Patroni automatically promotes a replica within seconds. HAProxy (`db`) sits in front of the cluster, continuously querying Patroni's health API to expose consistent endpoints (`db:5432` for writes, `db:5433` for reads) so application services require no configuration changes during a failover.

Redis cache and Sentinel. Redis caching uses a primary-replica architecture paired with three Redis Sentinels across separate nodes (quorum of 2). The application dynamically discovers the active master through Sentinel, enabling automatic failover if the primary node goes offline.

Moodle LMS. Moodle 4.5 LTS is deployed using PHP 8.3 Apache with two replicas. On initial launch, an automated script runs the command-line installer, utilizing a filesystem lock in the shared volume to prevent race conditions between containers. User sessions are persisted in MariaDB, allowing any replica to serve incoming requests interchangeably. A dedicated container (`moodle-cron`) runs scheduled tasks in the background.

Shared storage (NFS). Because Docker Swarm restricts privileged containers on worker nodes, we deploy **NFS-Ganesha** in user space on the `atenea` node. It exports shared volumes for Moodle data and certificates using only standard Linux capabilities, enabling seamless multi-node file sharing without requiring kernel-level NFS modules on the host.

Backups and observability. See Sections 7 and 8.

Scope of production. In the context of this project, "production" represents the complete multi-node Swarm architecture running on local Docker-in-Docker nodes. Because it is evaluated locally rather than on a public cloud provider, services use `localhost` subdomains and self-signed certificates.

4 Scalability and high availability

4.1 Replicas

The project guidelines require all services to be scalable, excluding periodic jobs and non-distributed DBMS. Stateless components (`frontend`, `core`, `static`, `db`, `smtp`, `grafana`, `moodle`, and `prometheus`) are deployed with two replicas and can be dynamically scaled using `docker service scale`. Prometheus automatically discovers active replicas via Docker Swarm DNS (`tasks.<service>`).

Services intentionally kept at a single replica include periodic background jobs (`letsencrypt`, `backup`, `moodle-cron`, and `auto`, where duplicate runs would be redundant), single-host storage (`nfs`), developmental tooling (`mailpit`), and `moodle-db`. The primary stateful layers (PostgreSQL and Redis) are not scaled naively, but are deployed as high-availability clusters with automated leader election and replication.

4.2 Failures

Table 2 summarizes the system resilience matrix, categorizing the direct impact, mitigation mechanism, and recovery process across service-level and node-level outages.

Failure Target	Direct Impact	Mitigation Mechanism	Recovery
Service / Container Failures			
Frontend replica	None	Swarm routing mesh directs traffic to survivor	Automatic
Core / static replica	None (transparent)	Nginx serves stale cache; Swarm reschedules	Automatic
Redis master	Brief cache miss spike	Sentinels promote replica; app degrades gracefully	Automatic (<5 s)
Postgres leader	Brief write pause	Patroni elects replica; HAProxy reroutes traffic	Automatic (<3 s)
Node Outages			
`front1` or `front2`	None	Routing mesh routes via surviving front node	Automatic
`app1` or `app2`	Reduced capacity	App2 serves; Patroni/Redis failover; etcd 2/3 quorum	Automatic
`stats`	Metrics interrupted	Core app unaffected; etcd/Patroni maintain 2/3 quorum	On node restart
`atenea`	Moodle & NFS down	Core app unaffected; TLS certs cached in Nginx	On node restart
`backup`	Backup jobs pause	User-facing services completely unaffected	On node restart

Table 2: System resilience and failure recovery matrix.

4.3 Updates

Updates are applied one container at a time (`parallelism: 1`, `order: stop-first`) with automated rollback upon error (`failure_action: rollback`). Each custom image includes a defined `HEALTHCHECK`, guaranteeing that an updated replica is verified healthy before the deployment progresses to the next node. Furthermore, stateless services enforce `max_replicas_per_node: 1`, ensuring replicas reside on distinct physical hosts so that a single host failure never compromises service availability.

5 Deployment

Local environment (one machine, no TLS):

```
docker compose pull && docker compose up -d
```

Production (seven nodes; `scripts/sim-swarm.sh up` does all this on one computer):

```
# once: build and publish the images
docker login && scripts/publish.sh

# on a manager, after the 7 nodes joined the swarm
scripts/label-nodes.sh # labels the nodes, prints the NFS_SERVER line
export NFS_SERVER=<private IP of atenea>
docker stack deploy -c production.yml lab1
```

All the images are in one public Docker Hub repository, `omarcornejo/lab1grouph`, with one tag per service (`<service>-<version>`), so no registry has to be started before deploying.

6 Access, users and passwords

All the passwords are Docker secrets, files of the `secrets/` folder, and all of them have the value `groupH`, a deliberate simplification of the laboratory (they would be changed before a real deployment). The only exception is the Moodle administrator, because Moodle refuses passwords that do not follow its policy (upper and lower case, digit and symbol).

6.1 Web addresses

Service	Local (`compose.yml`)	Production (`production.yml`)	Login
Reviews application	`http://app.localhost:8080`	`https://app.localhost`	none
Moodle (Atenea)	`http://lms.localhost:8080`	`https://lms.localhost`	yes
Grafana	`http://grafana.localhost:8080`	`https://grafana.localhost`	yes
Mailpit (captured mails)	`http://mail.localhost:8080`	`http://mail.localhost:8080`	none

Table 3: Web addresses. In production the certificate is self-signed, so the browser shows a warning that has to be accepted; port 80 redirects to 443.

Prometheus, Patroni, etcd, HAProxy and the exporters are not published: they are reached from inside the stack (for example, `docker exec` on the container).

6.2 Users and passwords

Service	User	Password	Secret file / note
Grafana	`admin`	`groupH`	`grafana_admin_password`
Moodle	`admin`	`groupH-2026`	`moodle_admin_password`
PostgreSQL, administrator	`postgres`	`groupH`	`pg_admin_password`; used only by Patroni
PostgreSQL, application	`app`	`groupH`	`app_db_password`; owner of database `app`
PostgreSQL, Grafana	`grafana`	`groupH`	`grafana_db_password`; owner of database `grafana`
PostgreSQL, replication	`replicator`	`groupH`	`replicator_password`
PostgreSQL, backups	`backup`	`groupH`	`backup_db_password`; read only
PostgreSQL, monitoring	`exporter`	`groupH`	`exporter_db_password`; `pg_monitor`
MariaDB (Moodle)	`moodle`	`groupH`	`moodle_db_password`
MariaDB, backups / monitoring	`backup`, `exporter`	`groupH`	same secrets as in PostgreSQL; read only
Backup encryption	—	`groupH`	`backup_passphrase` (AES-256)
Self-signed TLS key	—	`groupH`	variable `CERT_PASSWORD`
Feeder token (`/api/ratings`)	—	`groupH`	`feed_token`, bearer token
Docker Hub (pull)	—	—	public repository, no credentials

Table 4: Users and passwords of the project.

The reviews application needs no login: anybody can read and write reviews. In both environments the notification mails go to Mailpit, a mail catcher with a web page, so they can be read without any external provider. To send real mail, `RELAY_HOST`, `RELAY_PORT`, `RELAY_USER`, `RELAY_PASSWORD` and `SMTP_TLS` point the relay to a provider.

7 Data management and backups

Databases are not on NFS (locking and network cuts risk corruption). The backup service lives on a node that is neither an app node nor the NFS node, with its own disk, so no live data shares a

Data	Volume	Location
PostgreSQL (3 nodes), etcd	`pg-1..3`, `etcd-1..3`	local disk of app1, app2, stats
MariaDB, NFS data	`moodle-db`, `nfs-data`	local disk of atenea
moodledata, certificates	`moodledata`, `certs`, `acme`	NFS (atenea)
backups	`backups`	local disk of the **backup** node
Prometheus (2 replicas)	anonymous volume of each task	local disk of stats and backup
Redis	none	memory (cache, replicated)

Table 5: Where the data is stored.

node with its backup. Every night (03:30) it creates a folder with

- `app.dump.enc`, `grafana.dump.enc`: `pg_dump -Fc` from the *replicas* (`db:5433`, no load on the leader; falls back to the leader), as the read-only role `backup`;
- `moodle.sql.gz.enc`: `mariadb-dump -single-transaction` with a read-only MariaDB account;
- `moodledata.tar.gz.enc`: moodledata without caches and sessions;
- `SHA256SUMS`.

Every file is encrypted with AES-256-CBC, PBKDF2 (`openssl enc`) and the passphrase of a Docker secret. The folder is written as `.partial` and renamed at the end, so an interrupted backup is never mistaken for a good one. Folders older than 14 days are removed.

Automatic restore test. Every Sunday (and on demand with `docker exec <backup> /restore-test.sh`) the latest backup is verified: checksums, decryption, restoration of the application dump into a throw-away PostgreSQL started inside the container and of the Moodle dump into a throw-away MariaDB, and a count of rows and tables. The result is published as metrics (`backup_restore_test_ok`, rows, tables) that Prometheus scrapes, with alerts if the last backup is older than 25 h or the test fails. The result is the number of reviews and of Moodle tables recovered.

Restoring: `scripts/restore.sh <folder> app|grafana|moodle|moodledata`.

8 Observability

Prometheus uses DNS service discovery (`tasks.<service>`) so scaled replicas are scraped without changes. Table 6 lists the sources.

Source	Watches	What we look at
`core`	the application	requests per replica; reviews posted; cache hits and misses
nginx exporter (inside the frontend image)	each frontend replica	requests per second; active connections
postgres_exporter (one per node) and Patroni	the three PostgreSQL nodes	connections; transactions; replication lag; which node is the leader
redis_exporter	Redis master and replica	role; connected replicas; hit ratio; memory
mysqld_exporter	MariaDB of Moodle	connected threads; queries per second
blackbox_exporter	frontend, core, static, Moodle and Grafana	availability and response time as a user sees them (this is how Moodle is monitored)
cAdvisor and node_exporter	every node	CPU and memory per service and per host
backup service (port 9199)	the backup node	age, size and duration of the last backup; result of the restore test

Table 6: Metric sources.

The dashboard (as code, in git) has rows for overview, containers, nginx, PostgreSQL, Redis, MariaDB/Moodle and backups; the queries use `label_replace` so the same panel works with the swarm and the compose labels of cAdvisor. The Infinity datasource (the "?") shows the average and the ratings captured by the feeder straight from `/api/stats`. Rules in `rules.yml`: target or site down, replication lag, Postgres or Redis without exactly one leader/master, old backup, failed restore test. Moodle, which has no metrics endpoint, is covered by the probe of its login page and by the MariaDB exporter, which draws the arrow Grafana→atenea of the diagram.

9 Security

- Only nginx is exposed; port 80 serves ACME challenges and redirects to HTTPS (TLS 1.2/1.3, HSTS). Moodle, Grafana and the app are reached only through it.
- **Database roles with least privilege**: `app` owns database `app` and `grafana` owns its own; `replicator` (created by Patroni) can only replicate; `backup` has `pg_read_all_data`; `exporter` has `pg_monitor`; nobody uses the superuser, and `CONNECT` is revoked from `PUBLIC`. MariaDB has separate read-only `exporter` and `backup` accounts.
- Passwords are Docker secrets mounted in `/run/secrets`. All overlay networks are encrypted (IPsec) and each stack is isolated in its own network.
- The SMTP relay is not published; `/metrics` is blocked in nginx. The only other published page is the mail catcher (`mail.localhost:8080`), plain HTTP and without login, which is acceptable in the laboratory because it only holds notification mails.
- Backups are encrypted and verified.

10 Test cases

Each case says what to do, where to look and what must be seen. Commands are run from the project folder (local environment) or on the manager node, `front1` (production, `docker exec front1 ...`).

10.1 Local environment

Table 7: Test cases of the local environment.

	What we do	Where we look	Expected
L1	Publish a review in `app.localhost:8080`	`docker compose logs core | grep "POST /comment"`	status `302`; the review appears at the top and the counter grows
L2	(same review)	`docker compose logs smtp | grep status=sent`; page `mail.localhost:8080`	`status=sent`, and the mail "New review (n/5)" is listed
L3	Create a course in Atenea as `admin` (Site administration → Courses)	`docker compose logs moodle | grep "POST /course/edit.php"`; `docker compose exec moodle-db mariadb -umoodle -pgroupH moodle -e "select shortname from mdl_course"`	`303`; the course is in the table
L4	Stop the Postgres leader: `docker compose kill pg-1` (the leader)	`docker compose exec pg-2 patronictl -c /etc/patroni.yml list`; `docker compose logs pg-2 | grep promoted`	another node is *Leader*; "promoted self to leader"; a new review is still accepted
L5	Stop the Redis master: `docker compose stop redis-1`	`docker compose logs sentinel | grep switch-master`	`+switch-master mymaster redis-1 6379 redis-2 6379`; the page keeps working
L6	Backup and restore test	`docker compose exec backup /backup.sh`; `docker compose exec backup /restore-test.sh`	`backup ok: /backups/...`; `restore test ok: n comments, 494 moodle tables`

	What we do	Where we look	Expected
L7	More replicas: `docker compose up -d -scale core=3`	several requests; header `X-App-Node`	the node name changes between requests
L8	Open Grafana, dashboard *Lab1*	panels of PostgreSQL, Redis, nginx, backups	data in every panel; one Patroni node with value 1

10.2 Production environment

Table 8: Test cases of the production environment.

	What we do	Where we look	Expected
P1	Deploy the stack	`docker node ls`; `docker service ls`; `docker stack ps lab1`	7 nodes *Ready*; every service at its replicas; each task on its node
P2	Open `https://app.localhost`	browser; `curl -k -D- https://app.localhost/`	the page loads after accepting the certificate; `X-Front-Node` and `X-App-Node` alternate between requests
P3	Publish a review	`docker service logs lab1_core | grep "POST /comment"`	`POST /comment ... 302` on one of the two core replicas
P4	(same review)	`http://mail.localhost:8080`; `docker service logs lab1_smtp | grep status=sent`	the mail is listed; `relay=mailpit[...]:1025 ... status=sent`
P5	Create a course in Atenea	`https://lms.localhost`; `docker service logs lab1_moodle | grep course/edit.php`; table `mdl_course` in the `lab1_moodle-db` container	`POST /course/edit.php ... 303`; new row (for example `PRUEBA1`)
P6	Lose a whole node: `docker stop app2`	`docker node ls`; `patronictl list`; `curl -k https://app.localhost/`	`app2` *Down* in ≈13 s; a replica becomes leader; the page answers 200 and a new review is accepted after the election (at most the 30 s `ttl`)
P7	Kill the Postgres leader container: `docker kill <pg-N container>`	`docker service logs lab1_pg-N | grep promoted`; `patronictl list`	"promoted self to leader because I had the session lock"; timeline (TL) grows by one; swarm restarts the container as a replica; no review is lost
P8	Kill the Redis master container	`docker service logs lab1_sentinel-1 | grep switch-master`	`+switch-master mymaster redis-1 6379 redis-2 6379` in about 2 s; the page keeps working
P9	Rolling update: `docker service update -force lab1_frontend`	a request every half second to `https://app.localhost/`	replicas replaced one by one; no request fails
P10	Backup on the backup node	`docker exec <lab1_backup container> /backup.sh` and `/restore-test.sh`	`backup ok ... source: db:5433`; `restore test ok`
P11	Restore the application database from a backup	write a review after the backup, then `scripts/restore.sh <folder> app` on the backup node; count the reviews	the review written after the backup disappears and the earlier ones are back; the tables keep their owner and the application keeps working
P12	Open Grafana	`https://grafana.localhost`, dashboard *Lab1*; Prometheus on the stats and backup nodes	36 targets *UP* in each Prometheus; panels with data

11 Results

11.1 Local environment

The whole stack starts with two commands (`docker compose pull` and `docker compose up -d`) and in less than a minute the 29 containers are healthy (Moodle needs about three minutes the first time, because it installs itself). The application works end to end: a review is stored in PostgreSQL, appears at once on the page, generates a mail that can be read in Mailpit, and updates the dashboards of Grafana. The database and the cache survive the loss of their main node: stopping the PostgreSQL leader makes another node take its place in a few seconds (about 3 s if the container is stopped, about 28 s if it is killed), and stopping the Redis master promotes its replica; in both cases the application keeps accepting reviews and no review is lost. The backup is encrypted and the automatic restore test recovers the reviews and the 494 tables of Moodle.

11.2 Production environment (`production.yml`)

Production is the same stack spread over seven nodes. It runs on our own computer: the nodes are seven Docker-in-Docker containers (`scripts/sim-swarm.sh`), three of them swarm managers. All the services reach their replicas and each one is on the node that its label asks for. What we obtained:

- **Entry:** the three sites answer on 443 through both front nodes (self-signed certificate) and port 80 redirects; requests are spread over front1/front2 and app1/app2.
- **Loss of a whole node (app2):** swarm detects it in 13 s, a database replica is promoted, the page keeps answering and reviews are accepted again once the election ends; when the node returns, its database rejoins as a replica without intervention.
- **Loss of a front node:** the other front serves all the sites.
- **Rolling update:** the frontend was replaced replica by replica while a request was sent every half second, and none failed.
- **Mail, Atenea and Grafana:** the mail of a new review is listed in `http://mail.localhost:8080`, a course created in Atenea is stored (log and database) and both Prometheus replicas see all their targets up.
- **Backups:** made from a database replica on the backup node, encrypted, and the restore test passes (reviews and 494 Moodle tables).

12 Limitations

- **Production is simulated on one computer.** The seven nodes are Docker-in-Docker containers, so the behaviour with the latency of separate machines is not measured. It is not opened to the Internet and has no public domain: the certificate is self-signed (the Let's Encrypt logic is in the stack but stays idle).
- **Quorum.** PostgreSQL and etcd need two of their three nodes; if two fail at the same time, writes stop until one returns. The third PostgreSQL node shares its machine with the monitoring.
- **Single instances.** MariaDB, the NFS server and the mail catcher have one instance (the statement exempts DBMS); losing the atenea node stops Moodle until it returns.
- **Backups** are kept on one disk of one node; there is no off-site copy.
- **Security simplifications of the laboratory.** All the secrets are `groupH` and are committed; NFS uses `no_root_squash` (limited to private networks) and its port must be closed in a firewall; the mail catcher has no login.

13 Conclusions

The goal of the project was to turn the infrastructure of a company into a set of small services that can be started anywhere with a couple of commands, and to prove that it keeps working when something breaks. We have a complete system: a reviews application, a learning platform, monitoring, mail and backups, all packaged as images that anyone can download and run, on one machine for development or on seven nodes for production.

The part that taught us most was not starting the services but making them survive failures. In a distributed system the pieces have to find each other, agree on who is in charge and recover without anybody touching them, and many of the problems we found appeared only when we switched machines off on purpose. Testing those failures from the beginning, and not at the end, saved us from believing that something worked just because it had started.

The result is a system where losing a web server, an application server, the cache or the database leader goes unnoticed by the user, and where the data is protected by encrypted copies that are checked automatically by restoring them. It is not a public service and it keeps some simplifications that a real company would not accept, which are listed in the previous section, but the structure is the same one that would be used with real machines: the next steps would be to move it to servers with a real domain, to keep a copy of the backups outside, and to manage the passwords properly.

Work distribution

Member 1	basic stack: frontend, cache, Let's Encrypt, NFS
Member 2	core app and interface, static files, Redis cluster
Member 3	PostgreSQL cluster (Patroni) and roles, SMTP relay, feeder, backups
Member 4	Atenea (Moodle, MariaDB)
Member 5	stats stack, exporters, swarm cluster and documentation